TP

Objects Serialization [1]

- Java provides a mechanism, called object **serialization** where an object can be represented as a sequence of bytes that includes:
  - the object's data as well as
  - information about the object's type and the types of data stored in the object.
- After a serialized object has been written into a file, it can be read from the file and deserialized
  - The type information and bytes that represent the object and its data can be used to recreate the object in memory

Objects Serialization

- Classes **ObjectInputStream** and **ObjectOutputStream** are high-level streams that contain the methods for **serializing** and **deserializing** an object:
- The default mechanism will save:
  - **Object class name**
  - **Class signature**
  - **Values of all serializable fields**
- Methods use **serializing/ deserializing** :
  - **writeObject** - for writing
  - **readObject** - for restoration

ObjectOutputStream Class

```
ObjectOutputStream out = new ObjectOutputStream(fluxPrimitiv);
out.writeObject(referintaObiect);
fluxPrimitiv.close();


FileOutputStream fos = new FileOutputStream("test.ser");
ObjectOutputStream out = new ObjectOutputStream(fos);
out.writeObject("Ora curenta:");
out.writeObject(new Date());
out.writeInt(12345);
out.writeDouble(12.345);
out.writeBoolean(true);
fos.close();
```

- **Exceptions**:
  - **IOException**, **NotSerializableException**, **InvalidClassException**

ObjectInputStream Class

```
ObjectInputStream in = new ObjectInputStream(fluxPrimitiv);
Object obj = in.readObject();
fluxPrimitiv.close();

FileInputStream fis = new FileInputStream("test.ser");
ObjectInputStream in = new ObjectInputStream(fis);
String mesaj = (String) in.readObject();
Date data = (Date) in.readObject();
int i = in.readInt();
double d = in.readDouble();
boolean b = in.readBoolean();
fis.close();
```

- Exceptions
  - IOException, ClassNotFoundException

Serializable interface

- An object is serialized if the class of which it is part implements the **Serializable** interface

```
package java.io;
public interface Serializable {
  // empty
  }
public class ClasaSerializabila implements Serializable {
        // body of the class
    }
```

Control of serialization

- **Transient modifier** - ignore serialization

  transient private double temp;

  // Ignored by serialization

- **static modifier** - cancels the effect of the **transient** modifier

  static transient int N;

  // participate in serialization

static and transient modifiers

```
import java.io.*;
public class Test1 implements Serializable{
  int x=1; // Yes
  transient int y=2; // No
  transient static int z=3; // Yes
  static int t=4; // No
  public String toString () {
      return x + ", " + y + ", " + z + ", " + t; }
 }
```

Serializable - Example

- Let’s consider the Employee class

```
public class Employee implements java.io.Serializable {
   public String name;
   public String address;
   public transient int SSN;
   public int number;

   public void mailCheck() {
      System.out.println("Mailing a check to " + name + " " + address);
   }
}
```

- Notice that for a class to be serialized successfully, two conditions must be met
  - The class must implement the **java.io.Serializable** interface
  - All of the fields in the class must be serializable
    - » If a field is not serializable, it must be marked **transient**

Serializable - Example of Serializing an object

```
import java.io.*;
public class SerializeDemo {

   public static void main(String [] args) {
      Employee e = new Employee();
      e.name = "Reyan Ali";
      e.address = "Phokka Kuan, Ambehta Peer";
      e.SSN = 11122333;
      e.number = 101;

      try {
         FileOutputStream fileOut =
         new FileOutputStream("/tmp/employee.ser");
         ObjectOutputStream out = new ObjectOutputStream(fileOut);
         out.writeObject(e);
         out.close();
         fileOut.close();
         System.out.printf("Serialized data is saved in /tmp/employee.ser");
      } catch (IOException i) {
         i.printStackTrace();
      }
   }
}
```

Serializable - Example of Deserializing an Object

```
import java.io.*;
public class DeserializeDemo {

   public static void main(String [] args) {
      Employee e = null;
      try {
         FileInputStream fileIn = new FileInputStream("/tmp/employee.ser");
         ObjectInputStream in = new ObjectInputStream(fileIn);
         e = (Employee) in.readObject();
         in.close();
         fileIn.close();
      } catch (IOException i) {
         i.printStackTrace();
         return;
      } catch (ClassNotFoundException c) {
         System.out.println("Employee class not found");
         c.printStackTrace();
         return;
      }

      System.out.println("Deserialized Employee...");
      System.out.println("Name: " + e.name);
      System.out.println("Address: " + e.address);
      System.out.println("SSN: " + e.SSN);
      System.out.println("Number: " + e.number);
   }
}
```

This will produce the following result –

Output

```
Deserialized Employee...
Name: Reyan Ali
Address:Phokka Kuan, Ambehta Peer
SSN: 0
Number:101
```

Serializable - Example of Serializable/Deserializing an Object

- There are following important points to be noted:
  - The try/catch block tries to catch a **ClassNotFoundException**, which is declared by the **readObject**() method.
    - For a JVM to be able to deserialize an object, it must be able to find the bytecode for the class. If the JVM can't find a class during the deserialization of an object, it throws **a ClassNotFoundException**.
  - Notice that the return value of readObject() is cast to an **Employee** reference
  - The value of the SSN field was 11122333 when the object was serialized, but because the field is **transient**, this value was not sent to the output stream.
    - The SSN field of the deserialized Employee object is 0.

Serializable of inherited fields

- **Case 1: If superclass is serializable then subclass is automatically serializable**:
  - If superclass is Serializable, then by default every subclass is serializable.
    - » Hence, even though subclass doesn't implement Serializable interface(and if it's superclass implements Serializable), then we can serialize subclass object

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

// superclass A
// implementing Serializable interface
class A implements Serializable
{
    int i;

    // parameterized constructor
    public A(int i)
    {
        this.i = i;
    }

}
```

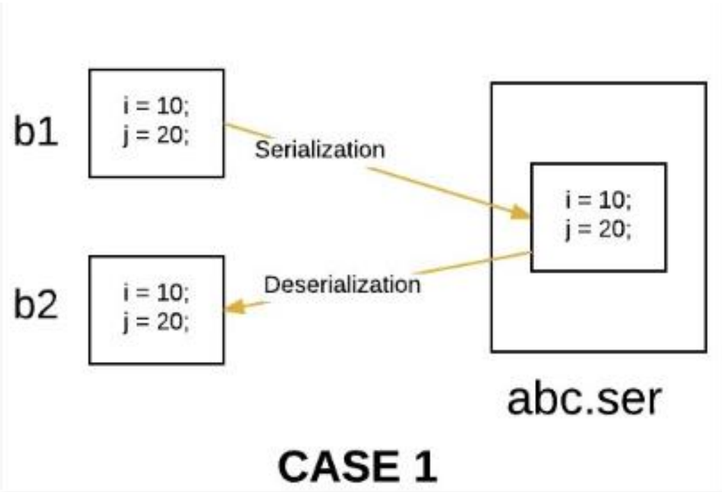
```
// subclass B
// B class doesn't implement Serializable
// interface.
class B extends A
{
    int j;

    // parameterized constructor
    public B(int i, int j)
    {
        super(i);
        this.j = j;
    }
}
```

Serializable of inherited fields – Case 1

```
// Driver class
public class Test
{
    public static void main(String[] args)
            throws Exception
    {
        B b1 = new B(10,20);

        System.out.println("i = " + b1.i);
        System.out.println("j = " + b1.j);

        /* Serializing B's(subclass) object */

        //Saving of object in a file
        FileOutputStream fos = new FileOutputStream("abc.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);

        // Method for serialization of B's class object
        oos.writeObject(b1);

        // closing streams
        oos.close();
        fos.close();

        System.out.println("Object has been serialized");

        /* De-Serializing B's(subclass) object */

        // Reading the object from a file
        FileInputStream fis = new FileInputStream("abc.ser");
        ObjectInputStream ois = new ObjectInputStream(fis);

        // Method for de-serialization of B's class object
        B b2 = (B)ois.readObject();

        // closing streams
        ois.close();
        fis.close();

        System.out.println("Object has been deserialized");

        System.out.println("i = " + b2.i);
        System.out.println("j = " + b2.j);
    }
}
```

Output:

```
i = 10
j = 20
Object has been serialized
Object has been deserialized
i = 10
j = 20
```



Serializable of inherited fields

- **Case 2: If a superclass is not serializable then subclass can still be serialized:**
  - Even though superclass doesn't implements **Serializable** interface, we can serialize subclass object if subclass itself implements **Serializable** interface
  - So we can say that to serialize subclass object, superclass need not to be serializable. But what happens with the instances of superclass during serialization in this case:
    - **Serialization:** At the time of serialization, if any instance variable is inheriting from non-serializable superclass, then JVM ignores original value of that instance variable and save default value to the file
    - **De-Serialization:** At the time of de-serialization, if any non-serializable superclass is present, then JVM will always invoke default(no-arg) constructor of that class.
      - So every non-serializable superclass must necessarily contain default constructor, otherwise we will get runtime-exception.

Serializable of inherited fields – Case 2

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;;

// superclass A
// A class doesn't implement Serializable
// interface.
class A
{
    int i;

    // parameterized constructor
    public A(int i)
    {
        this.i = i;
    }

    // default constructor
    // this constructor must be present
    // otherwise we will get runtime exception
    public A()
    {
        i = 50;
```

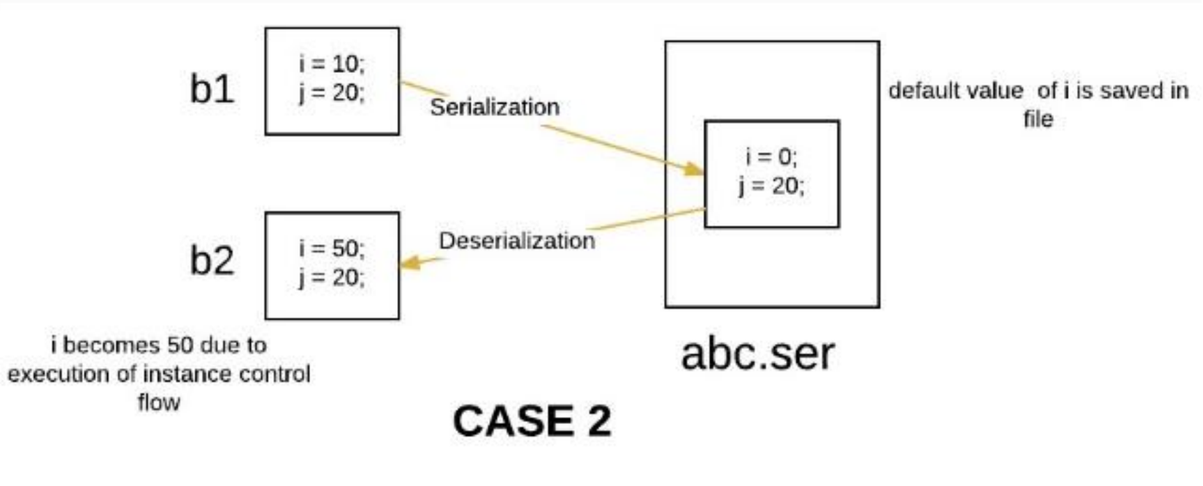
```
// subclass B
// implementing Serializable interface
class B extends A implements Serializable
{
    int j;

    // parameterized constructor
    public B(int i,int j)
    {
        super(i);
        this.j = j;
    }
}
```

Serializable of inherited fields – Case 2

```
public class Test
{
    public static void main(String[] args)
            throws Exception
    {
        B b1 = new B(10,20);

        System.out.println("i = " + b1.i);
        System.out.println("j = " + b1.j);

        // Serializing B's(subclass) object

        //Saving of object in a file
        FileOutputStream fos = new FileOutputStream("abc.ser");
        ObjectOutputStream oos = new ObjectOutputStream(fos);

        // Method for serialization of B's class object
        oos.writeObject(b1);

        // closing streams
        oos.close();
        fos.close();

        System.out.println("Object has been serialized");

        // De-Serializing B's(subclass) object

        // Reading the object from a file
        FileInputStream fis = new FileInputStream("abc.ser");
        ObjectInputStream ois = new ObjectInputStream(fis);

        // Method for de-serialization of B's class object
        B b2 = (B)ois.readObject();
```

Output:

```
i = 10
j = 20
Object has been serialized
A's class constructor called
Object has been deserialized
i = 0
j = 20
```



Serializable of inherited fields [2]

- **Case 3: If the superclass is serializable but we don’t want the subclass to be serialized**
  - There is no direct way to prevent subclass from serialization in java.
  - One possible way by which a programmer can achieve this is by implementing the *writeObject()* and *readObject()* methods in the subclass and needs to throw *NotSerializableException* from these methods.

Serialization and serialVersionUID

- We assume that we want to create an application that keeps track of the employees of a company

```
import java.io.*;
class Angajat implements Serializable {
  public String nume;
  public int salariu;
  private String parola;
  public Angajat (String nume , int salariu , String parola )
    {
          this.nume = nume;
          this.salariu = salariu;
          this.parola = parola;
  }
  public String toString () {
     return nume + " ( " + salariu + " ) " ;
    }
}
```

Serialization and serialVersionUID

- We consider an application that allows employees to be entered and saved into a file

```
import java.io.*;
import java.util.*;
public class GestiuneAngajati{
   //Lista angajatilor
  ArrayList ang = new ArrayList() ;
  public void citire () throws IOException {
      FileInputStream fis = null ;
      try{
           fis = new FileInputStream("angajati.ser") ;
           ObjectInputStream in = new ObjectInputStream(fis) ;
           ang = (ArrayList)in.readObject() ;
        }
   catch (FileNotFoundException e) {System.out.println("Fisierul nou ... " ) ;}
   catch (Exception e ) { System.out.println("Eroare la citirea datelor ... " ) ;
                                  e.printStackTrace();}
   finally{ if ( fis!= null )
                        fis.close() ;}
  System.out.println("Lista angajatilor :\ n " + ang );}
```

Serialization and serialVersionUID

```
public void salvare () throws IOException {
   FileOutputStream fos = new FileOutputStream ("angajati.ser");
   ObjectOutputStream out = new ObjectOutputStream(fos);
   out.writeObject(ang);
}
public void adaugare () throws IOException {
  BufferedReader stdin = new BufferedReader (new InputStreamReader (System.in)) ;
    while(true){
      System.out.println("Nume:") ;
      String nume = stdin.readLine() ;
      System.out.println( "Salariu:") ;
      int salariu = Integer.parseInt(stdin.readLine()) ;
      System.out.println( "Parola:") ;
      String parola = stdin.readLine() ;
      ang.add(new Angajat (nume, salariu, parola)) ;
      System.out.println ("Mai adaugati ? ( D / N )") ;
      String raspuns = stdin.readLine().toUpperCase() ;
    if (raspuns.startsWith("N"))
    break ;}}
```

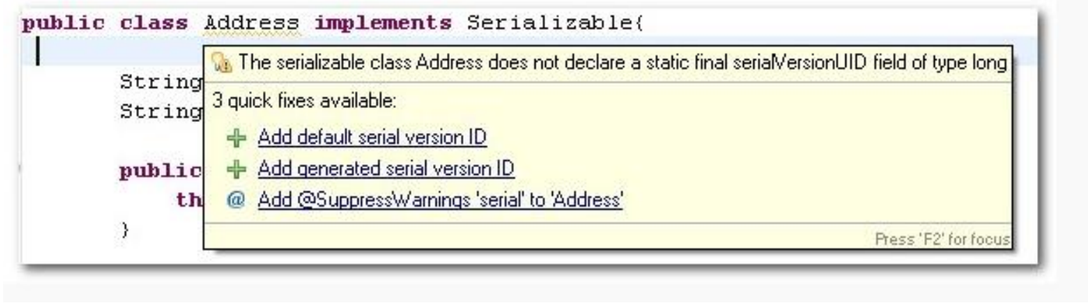
Serialization and **serialVersionUID**

```
public static void main(String args []) throws IOException {
  GestiuneAngajati app = new GestiuneAngajati () ;
   app.citire() ;
   // Adaugam noi angajati
   app. adaugare() ;
   // Salvam angajatii inapoi fisier
   app.salvare() ;
}
}
```

- After entering the employees into the file, the **Employee** class is modified by adding a new member variable, **address**
- When executing the application, an exception of the **InvalidClassException** type is throws when the employees are readed from the file because:
  - The Java serialization mechanism is very careful with the signatures of the serialized classes
  - For each serialized object, a 64-bit number is automatically calculated, which is a sort of "imprint" of the object class
    - This number, called **serialVersionUID**, is generated from various class information such as its member variables (but not only) and is saved in the serialization process together with the other data

Serialization and serialVersionUID

- So any significant change to the class, such as adding a new field, will cause its version number to change
- For the considered example the reason is clear that **serialVersionUID** of the previous class and new class are different
  - Actually if the class doesn't define **serialVersionUID**, it's getting calculated automatically and assigned to the class.
    - Java uses class variables, methods, class name, package etc to generate this unique long number.
  - If you are working with any IDE, you will automatically get a warning that "The serializable class Employee does not declare a static final **serialVersionUID** field of type long".
    - If you are using Eclipse, move your mouse over the serialization class.



Serialization and serialVersionUID

- Note that it’s not required that the serial version is generated from this program itself, we can assign this value as we want.
- It just need to be there to let deserialization process know that the new class is the new version of the same class and should be deserialized of possible.

Observer pattern

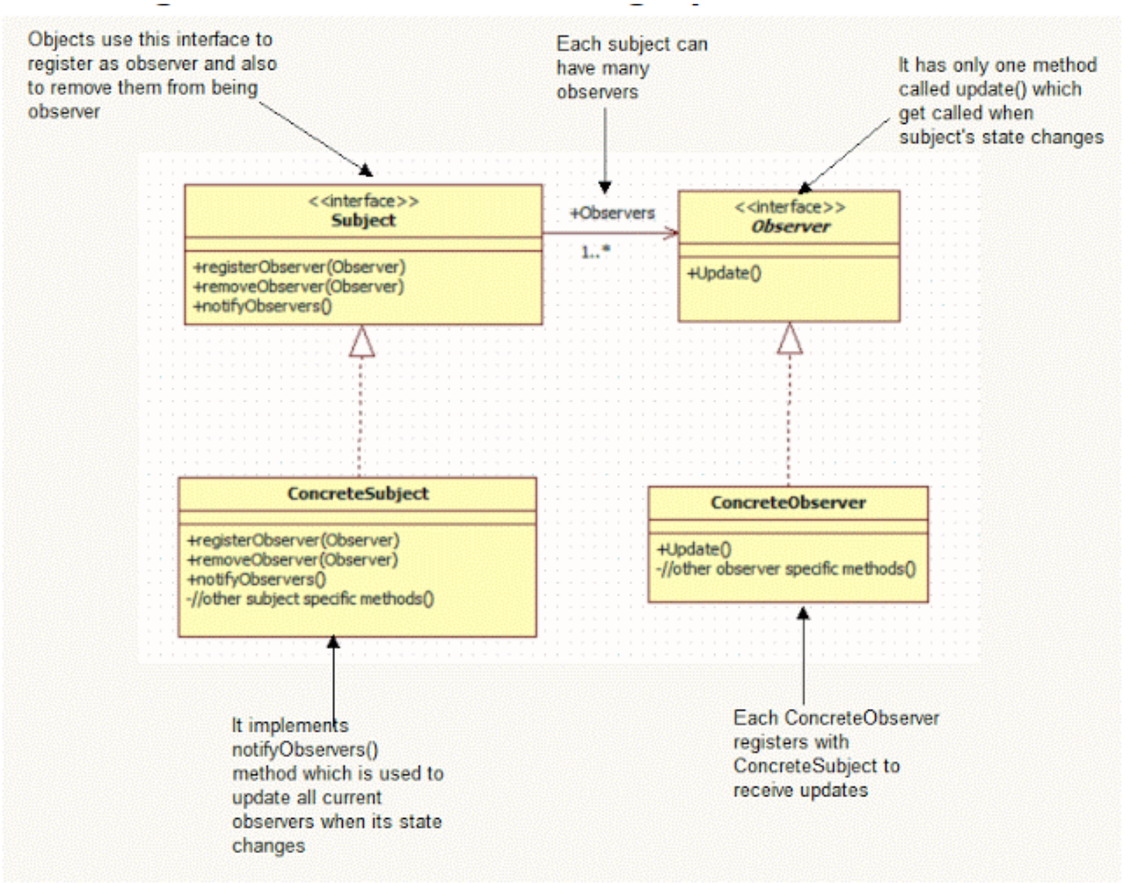
- As the name suggests it is used for observing some objects
- Observer watch for any change in state or property of subject
- Suppose you are interested in particular object and want to get notified when its state changes then you observe that object and when any state or property change happens to that object, it get notified to you
- You can think of observer as:
  - **Subject-Observer relationship**: Object which is being observed is refereed as **Subject** and classes which observe subject are called **Observer**
  - **Publisher-Subscriber relationship**: A publisher is one who publish data and notifies it to the list of subscribers who have subscribed for the same to that publisher. A simple example is Newspaper. Whenever a new edition is published by the publisher, it will be circulated among subscribers whom have subscribed to publisher.

Observer pattern

- **Some real life examples:**
  - You might have surfed "Flipkart.com-Online megastore". So when you search for any product and it is unavailable then there is option called "Notify me when product is available". If you subscribe to that option then when state of product changes i.e. it is available, you will get notification mail "Product is available now you can buy it". In this case, **Product** is subject and You are an observer.

Observer pattern

- **Components:**
  - **Subject**
    - » Knows its observers
    - » Has any number of observers
    - » Provides an interface to attach and detaching observer object at run time
  - **Observer**
    - » Provides an update interface to receive signal from subject
  - **ConcreteSubject**
    - » Stores state of interest to **ConcreteObserver** objects.
    - » Send notification to it's observer
  - **ConcreteObserver**
    - » Maintains reference to a **ConcreteSubject** object
    - » Maintains observer state consistent with subjects
    - » Implements **update** operation



Observer pattern

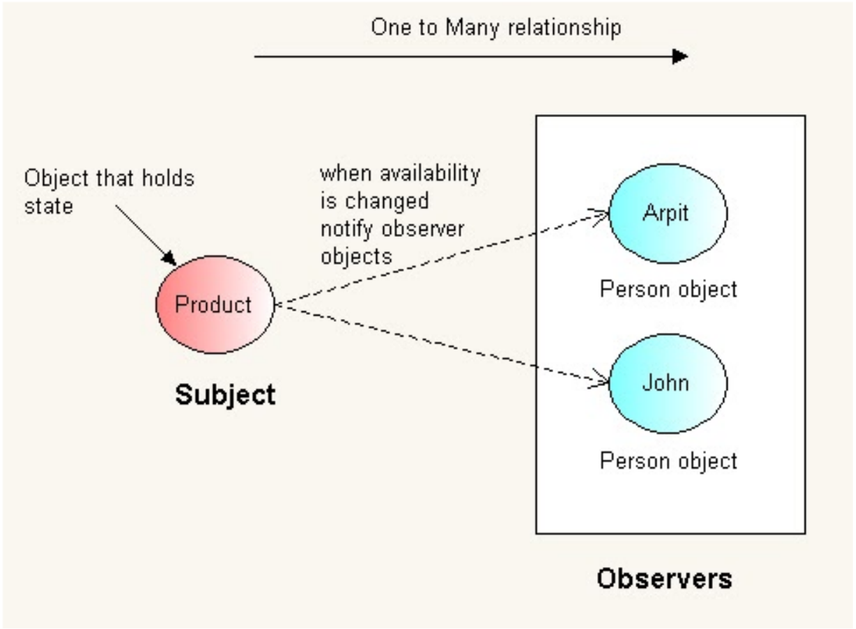
- **Java in-built API for observer pattern:**
  - The java API provides one **class** and one interface for implementing observer pattern
    - java.util.Observable - class
    - java.util.Observer - interface
- **java.util.Observable:**
  - For being observed, class must extend this class
    - The subclass becomes observable and override methods of **java.util.Observable** and other objects can "observe" changes in state of this object
  - **Methods:**
    - **addObserver(Observer o) :**add Observer to the list of observers for this subject
    - **deleteObserver(Observer o)** :delete Observers from the list of observers
    - **notifyObservers()** : notify all the observers if object has changed
    - **hasChanged()** :return true if object has changed
    - **setChanged()** :This method marks object has changed
    - **clearChanged()** :this method will indicate that subject has no changes or all the observers has been notified

Observer pattern

- **java.util.Observer:**
  - The class that performs the "observing" must implement the java.util.Observer interface. There is a single **method:**
    - **public void update(Observable obj, Object arg)**:This method is called whenever the observed object is changed and announces this fact by calling its notifyObservers() method.

Observer pattern

- **Example:**
  - You might have surfed "Flipkart.com-Online megastore". So when you search for any product and it is unavailable then there is option called "Notify me when product is available". If you subscribe to that option then when state of product changes i.e. it is available, you will get notification mail "Product is available now you can buy it".



Observer pattern

```
package org.arpit.javapostsforlearning;
import java.util.ArrayList;
import java.util.Observable;
import java.util.Observer;

public class Product extends Observable{

   private ArrayList<Observer> observers = new ArrayList<Observer>();
    private String productName;
    private String productType;
    String availability;


        public Product(String productName, String productType,String availability) {
                super();
                this.productName = productName;
                this.productType = productType;
                this.availability=availability;
        }

        public ArrayList<Observer> getObservers() {
                return observers;
        }
        public void setObservers(ArrayList<Observer> observers) {
                this.observers = observers;
        }
        public String getProductName() {
                return productName;
        }
        public void setProductName(String productName) {
                this.productName = productName;
        }
        public String getProductType() {
                return productType;
        }
        public void setProductType(String productType) {
                this.productType = productType;
        }

        public String getAvailability() {
                return availability;
        }
        public void setAvailability(String availability) {
               if(!(this.availability.equalsIgnoreCase(availability)))
               {
                       this.availability = availability;
                       setChanged();
                       notifyObservers(this,availability);
               }
        }

        public void notifyObservers(Observable observable,String availability) {
               System.out.println("Notifying to all the subscribers when product became available");
                for (Observer ob : observers) {
            ob.update(observable,this.availability);
        }

        }
```

Observer pattern

```
        public void registerObserver(Observer observer) {
                observers.add(observer);

        }

        public void removeObserver(Observer observer) {
                observers.remove(observer);

        }
}
```

Observer pattern

```
package org.arpit.javapostsforlearning;
import java.util.Observable;
import java.util.Observer;

public class Person implements Observer{

	String personName;


	public Person(String personName) {
		this.personName = personName;
	}
	public String getPersonName() {
		return personName;
	}
	public void setPersonName(String personName) {
		this.personName = personName;
	}

	public void update(Observable arg0, Object arg1) {
		System.out.println("Hello "+personName+", Product is now "+arg1+" on flipkart");

	}

}
```

Observer pattern

```
package org.arpit.javapostsforlearning;
public class ObserverPatternMain {

   /**
    * @Author arpit mandliya
    */
   public static void main(String[] args) {
    Person arpitPerson=new Person("Arpit");
    Person johnPerson=new Person("John");

    Product samsungMobile=new Product("Samsung", "Mobile", "Not available");

    //When you opt for option "Notify me when product is available".Below registerObserver method
    //get executed
      samsungMobile.registerObserver(arpitPerson);
    samsungMobile.registerObserver(johnPerson);

    //Now product is available
    samsungMobile.setAvailability("Available");

   }
}
```

https://dzone.com/articles/observer-design-pattern-java

Run it:

```
Notifying to all the subscribers when product became available
Hello Arpit, Product is now Available on flipkart
Hello John, Product is now Available on flipkart
```

HashTable

- Hashing is a fundamental concept of computer science
- In Java, efficient hashing algorithms stand behind some of the most popular collections we have available – such as the *HashMap* and the *HashSet*
- **Understanding How *hashCode()* Works**
  - Simply put, *hashCode()* returns an integer value, generated by a hashing algorithm
  - Objects that are equal (according to their *equals()*) must return the same hash code. **It's not required for different objects to return different hash codes.**

HashTable

- The general contract of *hashCode()* states:
  - Whenever it is invoked on the same object more than once during an execution of a Java application, *hashCode()* must consistently return the same value,
    - This value needs not remain consistent from one execution of an application to another execution of the same application
  - If two objects are equal according to the *equals(Object)* method, then calling the *hashCode()* method on each of the two objects must produce the same value
  - It is not required that if two objects are unequal according to the ***equals(java.lang.Object)*** method, then calling the *hashCode* method on each of the two objects must produce distinct integer results. However, developers should be aware that producing distinct integer results for unequal objects improves the performance of hash tables

HashTable

- **A Naive *hashCode()* Implementation**
  - We're going to define a sample *User* class that overrides the method's default implementation
  - The *User* class provides custom implementations for both *equals()* and *hashCode()*
  - Even more, there's nothing illegitimate with having *hashCode()* returning any fixed value
  - **However, this implementation degrades the functionality of hash tables to basically zero, as every object would be stored in the same, single bucket**
  - In this context, a hash table lookup is performed linearly and does not give us any real advantage

```
public class User {

    private long id;
    private String name;
    private String email;

    // standard getters/setters/constructors

    @Override
    public int hashCode() {
        return 1;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null) return false;
        if (this.getClass() != o.getClass()) return false;
        User user = (User) o;
        return id != user.id
          && (!name.equals(user.name)
          && !email.equals(user.email));
    }

    // getters and setters here
}
```

- **Improving the *hashCode()* Implementation**
  - Let's improve a little bit the current *hashCode()* implementation by including all fields of the *User* class so that it can produce different results for unequal objects:

```
@Override
public int hashCode() {
    return (int) id * name.hashCode() * email.hashCode();
}
```

  - This basic hashing algorithm is definitively much better than the previous one, as it computes the object's hash code by just multiplying the hash codes of the *name* and *email* fields and the *id*.
  - In general terms, we can say that this is a reasonable *hashCode()* implementation, as long as we keep the *equals()* implementation consistent with it.

HashTable

- **Standard *hashCode()* Implementations**
  - The better the hashing algorithm that we use to compute hash codes, the better will the performance of hash tables be
  - Let's have a look at a "standard" implementation that uses two primes numbers to add even more uniqueness to computed hash codes:

```
@Override
public int hashCode() {
    int hash = 7;
    hash = 31 * hash + (int) id;
    hash = 31 * hash + (name == null ? 0 : name.hashCode());
    hash = 31 * hash + (email == null ? 0 : email.hashCode());
    return hash;
}
```

HashTable

- While it's essential to understand the roles that *hashCode()* and *equals()* methods play, we don't have to implement them from scratch every time, as most IDEs can generate custom *hashCode()* and *equals()* implementations and since Java 7, we got an *Objects.hash()* utility method for comfortable hashing:
-

```
Objects.hash(name, email)
```

And Eclipse produces this one:

```
@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + ((email == null) ? 0 : email.hashCode());
    result = prime * result + (int) (id ^ (id >>> 32));
    result = prime * result + ((name == null) ? 0 : name.hashCode());
    return result;
}
```

HashTable

- Similarly, if we want Apache Commons Lang's HashCodeBuilder class to generate a hashCode() implementation for us, the commons-lang Maven dependency must be included in the pom file::

```
<dependency>
   <groupId>commons-lang</groupId>
   <artifactId>commons-lang</artifactId>
   <version>2.6</version>
</dependency>
```

- And *hashCode()* can be implemented like this:

```
1  public class User {
2      public int hashCode() {
3          return new HashCodeBuilder(17, 37).
4          append(id).
5          append(name).
6          append(email).
7          toHashCode();
8      }
9  }
```

HashTable

- Handling Hash Collisions
  - The intrinsic behavior of hash tables raises up a relevant aspect of these data structures: even with an efficient hashing algorithm, two or more objects might have the same hash code, even if they're unequal. So, their hash codes would point to the same bucket, even though they would have different hash table keys.
  - This situation is commonly known as a hash collision, and various methodologies exist for handling it
    - » Java's *HashMap* uses the separate chaining method for handling collisions:
  - **"When two or more objects point to the same bucket, they're simply stored in a linked list. In such a case, the hash table is an array of linked lists, and each object with the same hash is appended to the linked list at the bucket index in the array.**
  - Hash collision methodologies show in a nutshell why it's so important to implement *hashCode()* efficiently.

HashTable

- **Creating a Trivial Application**
  - To test the functionality of a standard *hashCode()* implementation, let's create a simple Java application that adds some *User* objects to a *HashMap*

```java
public class User {

    private long id;
    private String name;
    private String email;

    User(long i, String n, String e){
        id=i;
        name =n;
        email=e;
    }

    // standard getters/setters/constructors

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 31 * hash + (int) id;
        hash = 31 * hash + (name == null ? 0 : name.hashCode());
        hash = 31 * hash + (email == null ? 0 : email.hashCode());
       System.out.println("hashCode() called - Computed hash: " + hash);
        return hash;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null) return false;
        if (this.getClass() != o.getClass()) return false;
        User user = (User) o;
        return id != user.id
          && (!name.equals(user.name)
          && !email.equals(user.email));
    }

    // getters and setters here
}
```

HashTable

- **Creating a Trivial Application**
  - To test the functionality of a standard *hashCode()* implementation, let's create a simple Java application that adds some *User* objects to a *HashMap*

```
import java.util.HashMap;
import java.util.Map;

public class Application {

    public static void main(String[] args) {
        Map<User, User> users = new HashMap<>();
        User user1 = new User(1L, "John", "john@domain.com");
        User user2 = new User(2L, "Jennifer", "jennifer@domain.com");
        User user3 = new User(3L, "Mary", "mary@domain.com");

        users.put(user1, user1);
        users.put(user2, user2);
        users.put(user3, user3);
        if (users.containsKey(user1)) {
            System.out.print("User found in the collection");
        }
    }
}
```

```
hashCode() called - Computed hash: 1255477819
hashCode() called - Computed hash: -282948472
hashCode() called - Computed hash: -1540702691
hashCode() called - Computed hash: 1255477819
User found in the collection
```

Bibliography

- 1: https://www.tutorialspoint.com/java/java_serialization.htm
- https://www.geeksforgeeks.org/object-serialization-inheritance-java/